

Estratégia DeFi BTC - Blueprint Técnico

Objetivo

Tornar-se formador de mercado (liquidity provider) em pools DeFi de BTC, usando métricas e indicadores para ajustar timing, posição e exposição, visando ganhar taxas e oportunidades de arbitragem.

1. Escolha do ecossistema e infraestrutura

- DEXs com alto volume: Uniswap v3, Curve, PancakeSwap.
- Pools BTC tokenizado: WBTC, tBTC, renBTC.
- Redes com boas ferramentas: Ethereum, Arbitrum, Polygon.

Infraestrutura:

- API de preço: CoinGecko, Binance API, Kaiko.
- API on-chain: The Graph, Alchemy, Moralis.
- SDK do protocolo: Uniswap SDK.
- Armazenamento histórico: TimescaleDB ou InfluxDB.

2. Métricas de comportamento dos traders

- Funding Rate (Binance Futures)
- Open Interest
- Volume on-chain
- Taxa de utilização da pool
- Implied Volatility (Deribit)
- Arbitragem (1inch API)

3. Lógica central

1. Coleta de dados.
2. Processamento de indicadores compostos (Sentimento, Volatilidade, Oportunidade).
3. Regras:
 - Alta oportunidade + baixa volatilidade → range curto.
 - Alta oportunidade + alta volatilidade → range largo.
 - Baixa oportunidade → reduzir posição.
4. Ajuste de faixa no Uniswap v3.
5. Reinvestimento de fees.

4. Pseudocódigo simplificado

```
def coletar_dados():  
    funding = get_funding_rate("BTC")  
    oi = get_open_interest("BTC")  
    vol_onchain = get_dex_volume("BTC-WETH")  
    util_pool = get_pool_utilization("BTC-WETH")
```

```

iv = get_implied_volatility("BTC")
return funding, oi, vol_onchain, util_pool, iv

def calcular_indices(funding, oi, vol_onchain, util_pool, iv):
    sentimento = (funding * 0.6) + (oi * 0.4)
    volatilidade = iv
    oportunidade = (vol_onchain * 0.5) + (util_pool * 0.5)
    return sentimento, volatilidade, oportunidade

def decidir_liquidez(sentimento, volatilidade, oportunidade):
    if oportunidade > limiar_alto and volatilidade < vol_segura:
        return "range_curto", 0.8
    elif oportunidade > limiar_alto and volatilidade >= vol_segura:
        return "range_largo", 0.6
    else:
        return "reduzir", 0.2

while True:
    dados = coletar_dados()
    indices = calcular_indices(*dados)
    acao, capital = decidir_liquidez(*indices)
    executar_posicionamento(acao, capital)
    sleep(intervalo_minutos)

```

5. Controle de riscos

- Stop-loss indireto.
- Diversificação.
- Monitoramento constante.
- Uso de múltiplas pools.

6. Pipeline e cadência

- Ingestão: APIs e Subgraph.
- Processamento: cálculo de indicadores.
- Decisão: ajuste de liquidez.
- Execução: transações Uniswap v3.
- Monitoramento: métricas e alertas.

7. Backtest & Simulação

- Usar histórico do Subgraph e preços para simular fees e impermanent loss.
- Otimizar parâmetros (thresholds, ranges).

8. Regras Uniswap v3

- OPP alto + VOL baixo: faixa curta.
- OPP alto + VOL alto: faixa larga.
- OPP baixo: retirar liquidez.

9. Referências

- Uniswap v3 SDK e Subgraph.
- CoinGecko API.
- Binance Futures API.
- Deribit API.
- Alchemy.
- 1inch API.